

TDV 2215 NVRAM Complete Technical Documentation

Hardware Overview

ER3400 EAROM Chip Specifications

- **Manufacturer:** General Instrument (later acquired by Microchip)
- **Type:** 4096-bit Electrically Alterable ROM (EAROM)
- **Organization:** 1024 × 4-bit storage locations
- **Technology:** Early EEPROM predecessor with erase/write capability
- **Package:** 24-pin DIP package
- **Addressing:** 10-bit address bus (A0-A9) for 1024 locations
- **Data Width:** 4-bit nibbles (D0-D3 only)

Physical Connection to TDV 2215 System

Address Bus Connection

- **A0-A9:** Connected to CPU address bus bits 0-9
- **A10:** Special function - connects to **C1 control input**
- **Address Range:** CPU addresses 0x6000-0x67FF map to ER3400

Data Bus Connection

- **D0-D3:** Connected to CPU data bus bits 0-3 (4-bit data)
- **D4-D7:** **NOT connected to ER3400** - handled by system logic
- **D7 Hardware Logic:** System **hardcodes D7=1** during NVRAM reads

Control Signal Connections

- **C0 (pin 21):** Connected to **/S1** signal (inverted R/W)
- **C1 (pin 22):** Connected to **A10** (address bit 10)
- **NVSEL Enable:** Controlled by **ControlPort1 bit 2** (I/O port 0x50)

Control Logic Truth Table

A10 (C1)	/S1 (C0)	R/W	ER3400 Mode
0	1	0	READ
0	0	1	WRITE
1	1	0	ACCEPT ADDRESS
1	0	1	ERASE

Memory Address Mapping

CPU to ER3400 Address Translation

```
CPU Address → ER3400 Internal Address
0x6000-0x63FF → 0x000-0x3FF (normal read/write, A10=0)
0x6400-0x67FF → 0x000-0x3FF (erase/accept address, A10=1)
```

Nibble Storage Pattern

- Lower nibble:** Stored at base address (e.g., 0x50)
- Upper nibble:** Stored at base + 0x200 (e.g., 0x250)
- 8-bit reconstruction:** `(upper_nibble << 4) | lower_nibble`

NVRAM Content Organization

Memory Layout Overview

ER3400 Address	CPU Address	Content Type	Usage
0x00-0x0A	0x6000-0x600A	Template Data	Configuration extraction patterns
0x50-0x51	0x6050-0x6051	Signature Bytes	System validation (both 0x80)
0x52-0x5E	0x6052-0x605E	Configuration Data	Terminal settings and parameters

Detailed Memory Map

Configuration Template Area (0x00-0x0A)

ER3400 Addr	CPU Addr	Template Value	Purpose
0x00	0x6000	0xE8	Display configuration 1 extraction pattern
0x01	0x6001	0xE8	Display configuration 2 extraction pattern
0x02	0x6002	0xE8	Display configuration 3 extraction pattern
0x03	0x6003	0xFF	System configuration extraction pattern
0x04	0x6004	0xF8	Serial port configuration extraction pattern
0x05	0x6005	0xAF	Interface configuration extraction pattern
0x06	0x6006	0x80	Memory configuration 1 extraction pattern
0x07	0x6007	0x80	Memory configuration 2 extraction pattern
0x08	0x6008	0x80	Memory configuration 3 extraction pattern
0x09	0x6009	0x80	Reserved configuration 1 extraction pattern
0x0A	0x600A	0x80	Reserved configuration 2 extraction pattern

Signature Validation Area (0x50-0x51)

ER3400 Addr	CPU Addr	Expected Value	Purpose
0x50	0x6050	0x80	Primary signature - system validity check
0x51	0x6051	0x80	Secondary signature - NVRAM initialization confirmation

Configuration Data Area (0x52-0x5E)

ER3400 Addr	CPU Addr	Configuration Type	Function
0x52	0x6052	Display Config 1	Screen resolution, timing parameters
0x53	0x6053	Display Config 2	Character sets, display modes
0x54	0x6054	Display Config 3	Color settings, intensity levels
0x55	0x6055	System Config	Boot options, system behavior flags
0x56	0x6056	Serial Config	Baud rates, protocol settings
0x57	0x6057	Interface Config	Keyboard layout, special function keys
0x58	0x6058	Memory Config 1	CRITICAL: Memory test range specification
0x59	0x6059	Memory Config 2	Additional memory parameters
0x5A	0x605A	Memory Config 3	Memory buffer configurations
0x5B	0x605B	Reserved Config 1	Future expansion
0x5C	0x605C	Reserved Config 2	Future expansion
0x5D	0x605D	Reserved Config 3	Future expansion
0x5E	0x605E	Checksum Control	Final validation byte

Configuration Processing System

Template-Based Bit Extraction

The system uses a sophisticated **template-driven configuration extraction** system:

- 1. **Template at 0x00** defines bit extraction pattern for **configuration at 0x52**
- 2. **Template at 0x01** defines bit extraction pattern for **configuration at 0x53**
- 3. **And so on for all 11 configuration parameters**

Bit Field Extraction Process

c

```
// Pseudocode for bit extraction
for (config_index = 0; config_index < 11; config_index++) {
    template_byte = nvram[0x00 + config_index];
    config_byte = nvram[0x52 + config_index];

    // Extract bit fields based on template pattern
    extracted_value = extract_bits(config_byte, template_byte);

    // Store in configuration RAM at 0x5F00 + config_index
    config_ram[0x5F00 + config_index] = extracted_value;
}
```

Checksum Validation System

Checksum Calculation Logic

1. **Initialize:** Set `nvram_checksum_accumulator` (0x5E01) to 0
2. **For each configuration block:**
 - Read memory range specified by configuration
 - Sum all bytes in the range
 - Add sum to global checksum accumulator
3. **Final validation:** Compare accumulator with expected value **0xAA**

Memory Range Processing

- **Memory Config values (0x58-0x5A)** specify **memory ranges to checksum**
- **CRITICAL:** These values determine what memory gets tested
- **Common failure:** Invalid memory ranges cause unmapped memory access

System Boot and Validation Sequence

1. NVRAM Hardware Test

- **Enable NVRAM:** Set ControlPort1 bit 2 (NVSEL = 1)
- **Read signatures:** Verify 0x6050 and 0x6051 both return 0x80
- **Result:** Pass/Fail affects subsequent boot sequence

2. Configuration Load Process

- **Read template data:** Load patterns from 0x6000-0x600A
- **Process configuration:** Extract settings from 0x6052-0x605E

- **Build config structure:** Store results in RAM at 0x5F00+
- **Validate integrity:** Checksum must equal 0xAA

3. Memory Range Validation

- **Read memory configs:** Values at 0x6058-0x605A specify test ranges
- **Perform memory tests:** Test RAM at specified addresses
- **Common failure point:** Invalid ranges (like 0x4800-0x57FF) cause crashes

Access Protocols

NVRAM Read Operation

```
assembly

; Enable NVRAM
LDA (0x5E10)      ; Load current ControlPort1 value
ORI 0x04          ; Set NVSEL bit (bit 2)
OUT (0x50)        ; Enable NVRAM access

; Read lower nibble
MOV A,(HL)        ; Read from 0x60xx (HL = address)
ANI 0x0F          ; Keep only lower 4 bits
MOV D,A           ; Save lower nibble

; Read upper nibble
INR H             ; Add 0x200 to address (for upper nibble)
INR H
MOV A,(HL)        ; Read upper nibble location
RAL               ; Rotate left 4 times to position upper nibble
RAL
RAL
RAL
ANI 0xF0          ; Keep only upper 4 bits
ORA D             ; Combine with lower nibble

; Disable NVRAM
LDA (0x5E10)      ; Restore original ControlPort1
OUT (0x50)        ; Disable NVRAM access
```

NVRAM Write Operation (with Erase)

assembly

```
; Enable NVRAM
LDA (0x5E10)
ORI 0x04
OUT (0x50)

; Erase sequence (if data different)
INR H          ; Add 0x400 to address (set A10=1)
INR H
INR H
INR H
MOV (HL),A     ; Write to erase address (triggers erase)

; Write new data
; (Write actual data to base address)

; Disable NVRAM
LDA (0x5E10)
OUT (0x50)
```

Factory Reset and Initialization

Factory Reset Function (0x3820)

- **Triggered by:** Configuration menu option 6 (after 0xED + 6)
- **Process:** Writes complete default configuration to NVRAM
- **Includes:** Proper signature bytes, valid configuration data, correct checksum

Default Configuration Values

c

```
// Recommended NVRAM initialization for working system
void InitializeNVRAM() {
    // Signature bytes (must be 0x80)
    nvram[0x50] = 0x0; nvram[0x250] = 0x8; // Creates 0x80
    nvram[0x51] = 0x0; nvram[0x251] = 0x8; // Creates 0x80

    // Template data (defines extraction patterns)
    nvram[0x00] = 0x8; nvram[0x200] = 0xE; // Creates 0xE8
    nvram[0x01] = 0x8; nvram[0x201] = 0xE; // Creates 0xE8
    nvram[0x02] = 0x8; nvram[0x202] = 0xE; // Creates 0xE8
    nvram[0x03] = 0xF; nvram[0x203] = 0xF; // Creates 0xFF
    nvram[0x04] = 0x8; nvram[0x204] = 0xF; // Creates 0xF8
    nvram[0x05] = 0xF; nvram[0x205] = 0xA; // Creates 0xAF
    nvram[0x06] = 0x0; nvram[0x206] = 0x8; // Creates 0x80
    nvram[0x07] = 0x0; nvram[0x207] = 0x8; // Creates 0x80
    nvram[0x08] = 0x0; nvram[0x208] = 0x8; // Creates 0x80
    nvram[0x09] = 0x0; nvram[0x209] = 0x8; // Creates 0x80
    nvram[0x0A] = 0x0; nvram[0x20A] = 0x8; // Creates 0x80

    // Configuration data (adjust memory configs to valid RAM ranges)
    nvram[0x52] = 0x0; nvram[0x252] = 0x5; // Creates 0x50 (valid memory)
    nvram[0x53] = 0xC; nvram[0x253] = 0x5; // Creates 0x5C (valid memory)
    nvram[0x54] = 0xF; nvram[0x254] = 0x5; // Creates 0x5F (valid memory)
    // ... etc for remaining configuration bytes
}
```

Common Issues and Solutions

Issue 1: Unmapped Memory Access (0x4800-0x57FF)

- **Cause:** Memory config values (0x58-0x5A) specify invalid RAM ranges
- **Solution:** Set memory configs to point to valid RAM (0x5000-0x5FFF range)

Issue 2: Checksum Failure (Error 0x19)

- **Cause:** Configuration data doesn't produce expected checksum 0xAA
- **Solution:** Use factory reset function or calculate correct checksum values

Issue 3: Signature Test Failure (Error 0x1A)

- **Cause:** NVRAM returns 0x00 instead of 0x80 for signature bytes
- **Solution:** Ensure NVSEL enable/disable works and D7 hardcoding is implemented

Issue 4: Configuration Menu Inaccessible

- **Cause:** System stuck in boot error loop, interrupts not properly configured
- **Solution:** Fix NVRAM issues first, then verify UART interrupt handling

Integration with Terminal Software

Main Terminal Loop Integration

- **Normal operation:** System reads NVRAM configuration at startup
- **Configuration changes:** Settings modified via 0xED configuration menu
- **Persistent storage:** All changes written back to NVRAM for next boot
- **Validation:** Every boot verifies NVRAM integrity before proceeding

Keyboard Configuration Processing

- **Special key sequences:** 0xED enters configuration mode
- **Menu navigation:** Numeric keys select configuration options
- **Factory reset:** Option 6 reinitializes entire NVRAM contents
- **Exit sequences:** 0xDB handles mode transitions and saves settings

This NVRAM system is fundamental to the TDV 2215's operation, storing not just configuration data but also defining the memory ranges used for system validation and the bit extraction patterns used for configuration processing.